

BAYESIAN OPPONENT MODELLING IN BRISCOLA

**Inferring an opponent's latent playing style from partially
observed Briscola games**

We model how an opponent chooses cards, not how to play optimally

Overall Pipeline

- **Generate Briscola games** → Simulate Briscola games from opponents with known latent styles.
- **Collect partial observations** → Keep only public information, observer hand and played card.
- **Infer the opponent's hidden style** → Use sequential hand belief and variational inference.
- **Evaluate prediction and recovery** → Compare inferred style with ground truth and compute posterior predictive score on test games.



Synthetic Data Generation

1. **Choose** a ground-truth **style** θ_{true}
2. **Simulate two-player Briscola games**
observed opponent: theta-softmax policy
observer player: random policy
3. **Store** only **partial observations**



Visible:

$$D = \{(I_t, c_t)\}_{t=1}^T$$

D = observed dataset

I_t = observer information before the move
(public state + observer hand)

c_t = card played by the opponent

Hidden:

H_t = opponent's hidden hand

θ = opponent's playing style

Theta-Softmax Opponent

For each candidate card x , compute **features**:

$$\phi(x, H_t, I_t)$$

Then compute a linear **score**:

$$s_{\theta}(x; H_t, I_t) = \theta^{\top} \phi(x, H_t, I_t)$$

The card is sampled with **softmax**:

$$p(c_t = x \mid H_t, I_t, \theta) = \frac{\exp(s_{\theta}(x; H_t, I_t)/\tau)}{\sum_{x' \in H_t} \exp(s_{\theta}(x'; H_t, I_t)/\tau)}$$

Features examples:

is trump
point value
wins current trick

Sequential Hidden-Hand Belief

We do not know the true opponent hand, so we build a set of **compatible hands**:

$$\mathcal{H}_t = \{\text{hands compatible with observer information}\}$$

We maintain a **belief**: $b_t(H; \theta)$ = probability that the opponent hand is H

At the **beginning** of a game, the belief is **uniform**: $b_1(H) = \frac{1}{|\mathcal{H}_1|}$

For a move $\mathbf{t}$, the **predictive likelihood** is:

$$\ell_t(\theta) = p(c_t \mid I_t, c_{<t}, \theta) = \sum_{H \in \mathcal{H}_t} b_t(H; \theta) p(c_t \mid H, I_t, \theta)$$

Sequential Belief Update

After observing opponent move,
we **update** the **belief**:

$$\bar{b}_t(H; \theta) = \frac{b_t(H; \theta) p(c_t \mid H, I_t, \theta)}{\ell_t(\theta)}$$

Then propagate to the **next move**:

$$b_{t+1}(H'; \theta) = \sum_{H \in \mathcal{H}_t} \bar{b}_t(H; \theta) T_t(H' \mid H)$$

The belief is **sequential**: it carries information from previous moves instead of restarting uniformly every time.

$\ell_t(\theta)$: probability of the observed card,

$\bar{b}_t(H; \theta)$: updated belief after seeing the card,

$T_t(H' \mid H)$: transition to the next possible hand after card removal/draw.

Sequential Likelihood

For each move, the probability of the observed opponent card at time t is:

$$\ell_t(\theta) = p(c_t \mid I_t, c_{<t}, \theta)$$

For **one game**:
$$p_{\text{seq}}(D_g \mid \theta) = \prod_{t=1}^{T_g} \ell_{g,t}(\theta)$$

For **several games**
we use the log form:

$$\log p_{\text{seq}}(D_{\text{train}} \mid \theta) = \sum_{g \in \text{train}} \sum_{t=1}^{T_g} \log \ell_{g,t}(\theta)$$

The likelihood measures how probable the observed move sequence is under a candidate playing style θ while updating the hidden-hand belief throughout the game.

Variational Inference

Posterior Approximation

We first define the Bayesian posterior over the latent playing style, then approximate it with a simpler variational distribution.

Bayesian posterior: $p(\theta \mid D_{\text{train}}) \propto p_{\text{seq}}(D_{\text{train}} \mid \theta)p(\theta)$

$$p(\theta) = \mathcal{N}(0, \sigma_0^2 I)$$

The exact **posterior** is not tractable, so we **approximate it** with a **mean-field Gaussian** over the style vector θ :

$$q_{\lambda}(\theta) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$$

We choose the variational posterior by **maximizing** the **ELBO**:

$$\mathcal{L}(\lambda) = \mathbb{E}_{q_{\lambda}(\theta)} [\log p_{\text{seq}}(D_{\text{train}} \mid \theta)] - \text{KL}(q_{\lambda}(\theta) \parallel p(\theta))$$

The expectation term is estimated with **Monte Carlo samples**, and the **reparameterization trick** makes these samples differentiable with respect to μ and σ .

$$\hat{\mathcal{L}}(\lambda) = \frac{1}{S} \sum_{s=1}^S \log p_{\text{seq}}(D_{\text{train}} \mid \theta^{(s)}) - \text{KL}(q_{\lambda}(\theta) \parallel p(\theta))$$

$$\epsilon^{(s)} \sim \mathcal{N}(0, I), \quad \theta^{(s)} = \mu + \sigma \odot \epsilon^{(s)}$$

Adam optimizes μ and $\log \sigma$ by minimizing the negative ELBO: $\text{loss}(\lambda) = -\hat{\mathcal{L}}(\lambda)$

Monte Carlo Posterior Prediction

After training, VI gives an approximate posterior over the opponent style: $q_\lambda(\theta)$

To evaluate on **test** games, we do not use only one style estimate. Instead, we average predictions over plausible styles.

1. We **sample style** vectors from the learned posterior.

$$\theta^{(s)} \sim q_\lambda(\theta), \quad s = 1, \dots, S$$

2. **Evaluate** each style on the test games

$$p_{\text{seq}}(D_{\text{test}} \mid \theta(s))$$

3. **Average** the test **likelihoods**

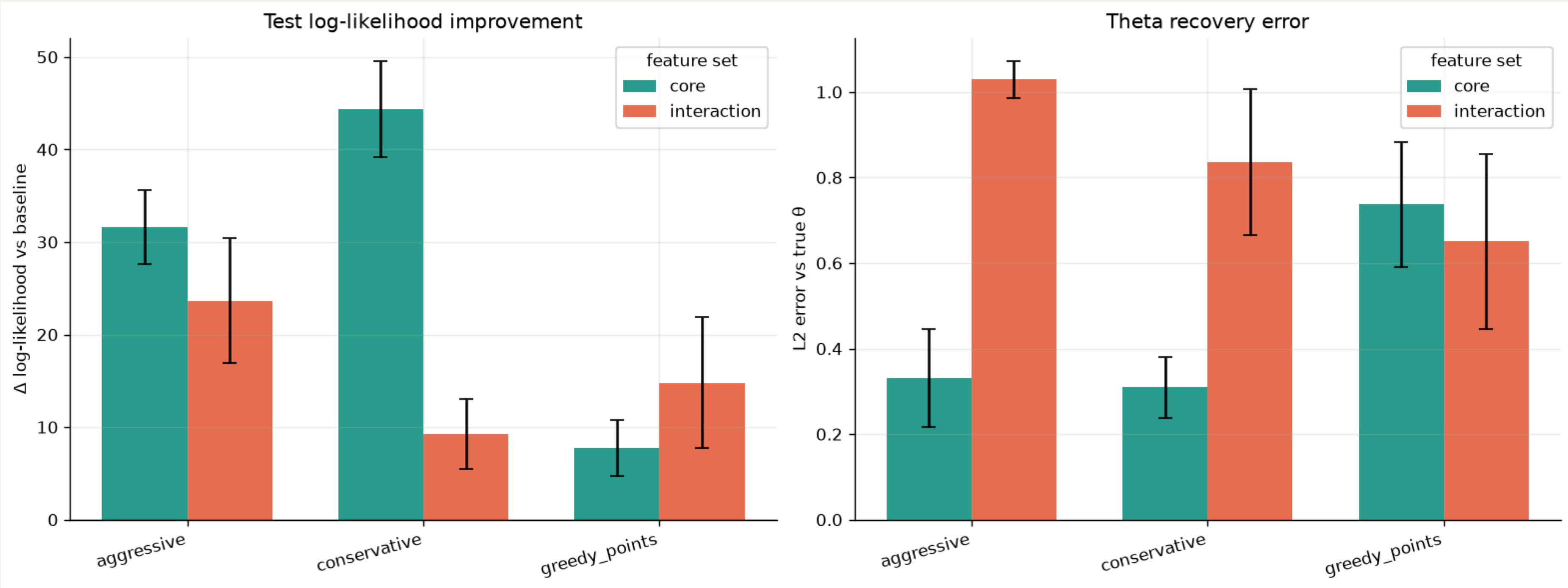
$$\log p(D_{\text{test}} \mid D_{\text{train}}) \approx \log \left[\frac{1}{S} \sum_{s=1}^S p_{\text{seq}} \left(D_{\text{test}} \mid \theta^{(s)} \right) \right]$$

Results

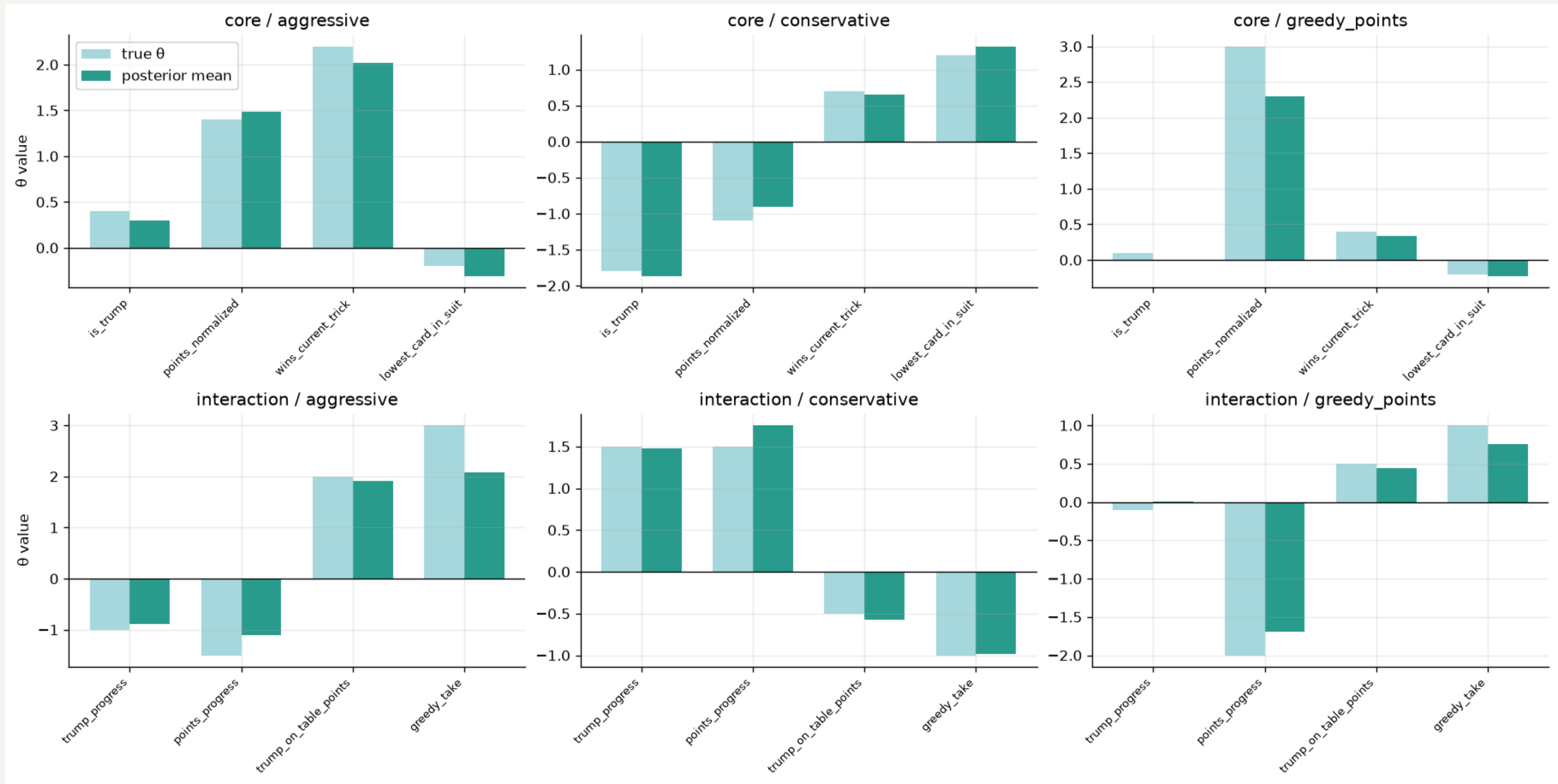
Core	theta L2 ↓	test Δ ↑	mean logp Δ ↑	prob x ↑	ELBO ↑
Aggressive	0.331	31.59	0.0632	1.065x	-3654.9
Conservative	0.309	44.38	0.0888	1.093x	-3609.7
Greedy_points	0.737	7.73	0.0155	1.016x	-3708.5

Interaction	theta L2 ↓	test Δ ↑	mean logp Δ ↑	prob x ↑	ELBO ↑
Aggressive	1.029	23.67	0.0473	1.048x	-3685.6
Conservative	0.836	9.24	0.0185	1.019x	-3710.5
Greedy_points	0.651	14.81	0.0296	1.030x	-3717.5

Core vs Interaction



Theta Recovery: True Style vs Posterior Mean



Next steps

- Use the learned inference module to build a **full Briscola playing agent**.
- Evaluate the model's impact on win rates by replacing synthetic profiles with unconstrained **human** or **self-play** data.
- Replace hand-written features with learned or automatically selected features.
- Explore richer posterior approximations.
- Compare VI with **alternative inference methods**, such as MCMC or particle filtering.



THANK YOU FOR YOUR ATTENTION!

<https://github.com/alinafogar/Briscola-PML>